



GHANA COMMUNICATION TECHNOLOGY UNIVERSITY

FACULTY OF ENGINEERING

ARDUINO ASSIGNMENT REPORT: SMART COUNTDOWN

Student Name:	Sheena Ama Anyawoe	Class/Group:	Group_B (Morning)
Index Number:	2526403292	Date:	June 24, 2026

1. Introduction & Objectives

This report outlines the implementation and functional analysis of an embedded system firmware designed using the Arduino framework. The primary objective of the program, titled "**SMART COUNTDOWN**", is to execute a structured micro-controller routine that pairs visual LED feedback with asynchronous Serial communication outputs.

By analyzing the structural components of the provided code file `Assignment1_Sheena.ino`, this report evaluates the code execution flow, hardware resource configuration, sub-routine performance parameters, and precise timeline behavior of the system.

2. Source Code Listing

The firmware implementation provided for analysis is structured as follows:

```
int startvalue = 5;
int pinled = 7;

void flashled(int times){
  for (int flashCount = 0; flashCount < times; flashCount++){
    digitalWrite(pinled, HIGH);
```

```

        delay(250);

        digitalWrite(pinled, LOW);
        delay(250);
    }
}

void setup() {
    pinMode(pinled, OUTPUT);

    Serial.begin(9600);
    Serial.println("=====SMART COUNTDOWN=====");
    for (int count = startvalue; count > 0; count = count - 1){

        Serial.print("count: ");
        Serial.println(count);

        flashled(count);
        delay(1000);
    }

    Serial.println("=====Countdown is complete=====");
}

void loop() {}

```

3. Comprehensive Task and Question Analysis

Task 1: Global Variable Initialization and Resource Mapping

Analysis & Core Questions addressed: What are the initial conditions of the script and how are micro-controller peripherals allocated?

The sketch defines two essential global variables at the entry point of the code:

- `int startvalue = 5;` Sets the initial state of the structural countdown loop. By defining this globally, the user can easily re-configure the lifespan of the counter without deep-diving into loop architectures.
- `int pinled = 7;` Explicitly allocates General Purpose Input/Output (GPIO) Pin 7 of the Arduino board to an external or internal LED indicator.

Hardware Impact: During compilation, these variables are placed in the SRAM memory space. Mapping pin definitions globally ensures structural consistency and read-access availability across all custom scopes (e.g., inside `flashled()` and `setup()`).

Task 2: Sub-routine Behavior and Timing Characteristics

Analysis & Core Questions addressed: How does the custom function operate, and what is its temporal profile per call?

The sub-routine `void flashled(int times)` provides a parameterizable behavior framework to produce physical pulses on Pin 7.

Within the function, a local `for` loop iterates exactly ***times*** number of cycles. Each individual cycle executes a full square-wave pulse consisting of:

1. Setting Pin 7 to a logic high state (HIGH / ~5V), pulling current through the LED.
2. Blocking processor operations for ***250 ext{ ms}*** using `delay(250)`.
3. Setting Pin 7 to a logic low state (LOW / 0V), stopping current flow.
4. Blocking processor operations again for ***250 ext{ ms}*** using `delay(250)`.

Timing Analysis Rule: Each standalone flash duration (T_{flash}) can be modeled mathematically as:

$$T_{\text{flash}} = t_{\text{high}} + t_{\text{low}} = 250 \text{ ms} + 250 \text{ ms} = 500 \text{ ms} = 0.5 \text{ s}$$

Therefore, for a variable argument n , the total function execution latency satisfies:

$$T_{\{ \text{ext}\{total\} \}}(n) = n \text{ times } 500 \text{ ext}\{ ms \}$$

Task 3: Operational Control Flow and Countdown Execution

Analysis & Core Questions addressed: Where does the main logic execute and how does the iteration progress?

Unconventionally, the primary system algorithm is embedded completely within the `setup()` block instead of `loop()`. Because `setup()` runs exactly once upon system reset, the countdown behaves as a one-shot task. The `loop()` function remains empty, which prevents any recursive behavior post-execution.

The control sequence decrements starting from 5 down to 1. The structural behavior and sequential output telemetry are tracked in the table below:

Iteration Index	Current Value (count)	Serial Output Telemetry	LED Flash Multiplier	LED Routine Active Duration	Post-Routine Delay
1	5	count: 5	5 Times	2500 ms (2.5 seconds)	1000 ms (1.0 second)
2	4	count: 4	4 Times	2000 ms (2.0 seconds)	1000 ms (1.0 second)
3	3	count: 3	3 Times	1500 ms (1.5 seconds)	1000 ms (1.0 second)
4	2	count: 2	2 Times	1000 ms (1.0 second)	1000 ms (1.0 second)
5	1	count: 1	1 Time	500 ms (0.5 second)	

Iteration Index	Current Value (count)	Serial Output Telemetry	LED Flash Multiplier	LED Routine Active Duration	Post-Routine Delay
					1000 ms (1.0 second)

Task 4: Serial Monitor Terminal Output Transcript

Analysis & Core Questions addressed: What will a user observe on the hardware monitor interface?

The system establishes connection via UART at 9600 Baud (bits per second). The literal print transcript observed sequentially in the Serial Monitor is as follows:

```
=====SMART COUNTDOWN=====
count: 5
count: 4
count: 3
count: 2
count: 1
=====Countdown is complete=====
```

4. Conclusion

The firmware successfully executes an intelligent, proportional-feedback countdown system. Through the utilization of modular code patterns (the `flashLed()` routine), the program avoids structural redundancy while accurately communicating system indices across both physical (LED status) and telemetry (Serial stream) channels. The timing budget balances accurately with predictable deterministic execution delays.